

DL India Core

Full workflow reference: what every file does, and how the dashboard runs end to end.

Repository	pangoriyaanigrah-maker/dl-india-core (private)
Live site	pangoriyaanigrah-maker.github.io/dl-india-core
Stack	One HTML file · vanilla JavaScript · sql.js (SQLite/WASM) · Python 3.12 + yfinance · GitHub Actions
Hosting	GitHub Pages — static hosting, no server behind the site itself

Contents

- &bThe core idea — two kinds of data, kept apart
- &bFile-by-file reference
- &bStatic site, dynamic app
- &bThe upload & save workflow
- &bHow a new holding gets priced
- &bThe nightly job, step by step
- &bThe six tabs
- &bKey formulas
- &bKnown limits
- &bIf something breaks

1. The core idea

Everything about how this project is built follows from one decision: the numbers a human judges and the numbers a market reports never mix.

Human data — you own it	Market data — fetched for you
Weight, full weight, target price, add level, conviction, thesis, strategy, sector, industry, analyst, cash, trades.	Current price, market cap, 52-week range, value/momentum/quality z-scores, benchmark sector & size weights, beta, drawdown.
Lives in <code>book.sqlite</code> . Arrives by Excel upload, you click Save Permanently.	Lives in <code>signals.json</code> . Arrives automatically, every weekday 6pm IST.

Nobody ever hand-types a price. Nobody ever downloads a thesis. That separation is why a bad upload can never corrupt years of price history, and a failed price fetch can never erase an analyst's work. When market data is missing for a holding, the dashboard says NO FEED — it never fills the gap with a guess.

2. File-by-file reference

<code>index.html</code>	The entire dashboard — markup, styling and logic in one file. No framework, no build step. Runs <code>sql.js</code> (SQLite compiled to WebAssembly) directly in the browser, so the database itself lives and runs client-side.
<code>book.sqlite</code>	Your book: holdings, targets, thesis, strategy, cash, trade ledger. An actual SQLite database file, not a plain JSON blob. Starts empty by design — nothing shows until a holdings file (and a trades file) is uploaded and saved.
<code>signals.json</code>	Last night's prices and factor scores, in plain JSON. Overwritten every weekday evening by the script below — never hand-edited, never touched by an upload.
<code>build_signals.py</code>	The price fetcher. Reads every ticker in <code>book.sqlite</code> , resolves each to a Yahoo Finance symbol, pulls 5 years of daily prices for the book plus ~500 comparison stocks (Nifty Smallcap 250 + Microcap 250), computes value/momentum/quality z-scores and the sector/size benchmark, writes <code>signals.json</code> .
<code>.github/workflows/update-signals.yml</code>	The schedule: runs <code>build_signals.py</code> automatically every weekday at 18:00 IST on GitHub's own machines. A manual "Run workflow" button exists for on-demand runs too.
<code>serve.py</code>	A tiny local server, used only when running the dashboard on your own PC. Serves the folder, and accepts one POST route that writes <code>book.sqlite</code> to disk — the one thing a static host (GitHub Pages) can never do.
<code>start_dashboard.bat</code>	Double-click launcher for local use: pulls the latest signals from GitHub, starts <code>serve.py</code> , opens the browser.
<code>reset_book.py</code>	The one and only way to wipe the book back to empty. Deliberately a separate script, not an in-app button — deleting data is not something the running dashboard can trigger on its own.

<code>make_test_xlsx.py</code>	Builds throwaway test fixtures from the real book, with every blank filled in and a Trades sheet added, for trying things safely without risking real data.
<code>.gitignore</code>	Keeps generated/throwaway files (the raw Excel import, test fixtures, editor config) out of version control, while <code>book.sqlite</code> and <code>signals.json</code> stay tracked as the real state of the project.

3. Static site, dynamic app

How it's **served**, and what it **does**, are two different questions with two different answers.

How it's served: static.

GitHub Pages hands out fixed files exactly as they sit in the repository. No server runs code per request, no backend, no database server sitting behind the site. That is precisely why deleting data had to move *out* of the app — a static site cannot run a privileged “delete” endpoint, because there is no server there to trust with it.

What runs in the browser: fully dynamic.

Once the page loads, JavaScript does real work locally: SQLite runs live in the tab via WebAssembly, every P&L figure and risk scenario is computed on the spot, and the page talks to the GitHub API directly to save. None of this needs a server — it is dynamic *client-side*, not server-side.

The one genuinely server-side piece: GitHub Actions.

Every weekday evening, GitHub runs `build_signals.py` on its own infrastructure — real server-side computation, decoupled from the website. The site only ever reads its output as a static file. Locally, `serve.py` is the one true dynamic server, and it only ever runs on your own machine via the launcher, never on the hosted link.

SUMMARY

Static site, dynamic app. Almost everything that needs to be “live” happens either in the browser or on a schedule elsewhere — nothing requires a server sitting behind the website itself.

4. The upload & save workflow

A deliberately simple, two-step shape: **Load** stages a file on screen with nothing persisted; **Save Permanently** is the only action that writes anything down, and it always asks for confirmation first.

01 Drop a holdings file

Parsed in the browser, shown as a preview table. Replaces the whole holdings snapshot — a holdings file represents the full current book, not a delta.

02 Drop a trades file

Parsed and shown as a preview. New rows are merged into the existing trade ledger, de-duplicated by (date, ticker, side, qty, price) — a trades file is a history that accumulates, never a replacement for what came before.

03 Click Load

Both files move into on-screen state. Nothing is written anywhere yet — safe to look around, switch tabs, even close the page without consequence.

04 Click Save Permanently

A confirm dialog states exactly how many holdings and trades are about to be written. Only on confirmation does the save actually happen.

05 Where it saves

Disk (`book.sqlite`, only possible when running locally via `serve.py`), then GitHub if a token is connected (a real commit, with full history), then the browser only as a last resort if neither of those is available.

WHY THIS SHAPE

The user should decide when data becomes permanent, not the app deciding it automatically. Uploading is exploration; saving is a deliberate act with its own confirmation step. Deleting works the same way in reverse — it happens outside the running app entirely, via `reset_book.py`, so a static, unattended page can never be tricked into wiping data on its own.

5. How a new holding gets priced

Add a ticker that was not in the book before, and here is exactly what happens, in order:

- 01 Right after upload**
Shows NO FEED — no price, no upside, no z-scores. Expected, not an error: the price-fetcher has not run for it yet.

- 02 After Save Permanently, connected with Actions permission**
The nightly job is triggered immediately via GitHub's workflow_dispatch API — priced within a few minutes.

- 03 After Save Permanently, not connected (or missing Actions permission)**
Waits for the next scheduled run — 18:00 IST, automatic, no action needed.

- 04 Every weekday 6pm IST regardless**
build_signals.py reads every ticker currently in the book and fetches it. A new holding is automatically included — nothing extra to configure.

THE ONE REAL LIMIT

Yahoo is tried as TICKER.NS, then TICKER.BO. For a genuinely obscure or very recently listed stock, neither may resolve, and it stays NO FEED indefinitely — fixed by setting a manual YfSymbol override for that holding.

6. The nightly job, step by step

- 01 Read the book**
Opens book.sqlite, reads every holding's ticker and optional YfSymbol override.

- 02 Resolve each symbol**
Tries TICKER.NS, falls back to TICKER.BO — catches SME names listed only on BSE.

- 03 Download five years of daily prices**
For every holding, plus roughly 500 comparison names from Nifty Smallcap 250 + Nifty Microcap 250 — the peer group a small/micro-cap book is actually measured against.

- 04 Score against that group**
Value, momentum and quality, each a cross-sectional z-score. A score only means something relative to a crowd; this is why the comparison universe is fetched at all.

- 05 Write and commit**
Saves signals.json, commits it, pulls-and-rebases before pushing so a concurrent save from the app never breaks the push, then pushes.

Timing note: GitHub's own scheduler is not perfectly punctual — a run can start anywhere from a few minutes to, occasionally, over an hour after the scheduled time, depending on load across GitHub's infrastructure. It always runs; it just is not a precision clock.

7. The six tabs

Tab	Shows
Overview	Total/realized/unrealized P&L, composition vs full-build target, staged buys and sells, four-way attribution (sector, momentum cohort, size, analyst), rule-based written insights.
Exposures	Factor table — book z-score, tilt, highest and lowest name per factor, vs the Smallcap+Microcap 250 cross-section. Size mix and sector active-weight vs benchmark.
Risk	Four historical stress scenarios (52-week-range reversion, giving up last month's gain, retesting the 52-week low, worst historical month repeating) — all backward-looking, no simulation. Beta, daily volatility, max drawdown per stock.
Performance	Realized/unrealized P&L by position on an average-cost basis, full position detail, complete trade log. Reads NO with no trades file — that is correct, not broken.
Positions	One row per holding: weight, a price rail from 52-week-low to target with the current price and any staged add-level marked, upside, factor z-scores, status badge. Tap a row for the thesis and strategy text.
Import	Upload holdings and trades, connect GitHub, Save Permanently.

8. Key formulas

Metric	Formula
Upside	$\text{target} \div \text{current price} - 1$
Active weight	$\text{your sector \%} - \text{benchmark sector \%}$
Realized / unrealized P&L	average-cost method, matching a standard Excel tracker
Momentum z-score	z of [(12-month – 1-month) + 6-month] price return, cross-sectional
Value z-score	z of cheapness on P/E (forward, else trailing), P/B, EV/EBITDA
Quality z-score	z of ROE, leverage and net margin, equally weighted
Portfolio beta	$\text{weighted stock beta} \times \text{equity fraction of NAV}$

Status badges, read in priority order: full weight of zero always means EXITING, regardless of upside. Otherwise: no price feed at all is NO FEED; upside at or below zero is TARGET HIT; under 10% remaining is NEAR TARGET; anything more is ON THESIS.

9. Known limits

Limit	Detail
A holding with no Target price	Shows no price rail at all — the rail's own axis is built from the target, so nothing (including an add-level marker) can be placed on it without one.
Sector must match the benchmark's vocabulary	The Sector column drives active-weight math and must use the benchmark's own taxonomy (fetched from Yahoo). A thematic label like “Oil & Gas Pipes” belongs in the separate Industry column instead, which is display-only.
A very recent listing	Needs a full year of price history for momentum, and a full 52-week range for several risk metrics. Shows blank fields until that history exists — correct behaviour, not a gap in the fetcher.
Benchmark composition drifts between rebalances	NSE fixes Smallcap/Microcap 250 membership twice a year; a constituent's real market cap can drift well past its official bucket in the meantime. The Size panel shows the true, current market caps — which is why it is not always a clean 100% small/micro split.
A GitHub token needs the right scope	Contents: Read and write saves the book. Actions: Read and write is also needed for the instant new-ticker price trigger — without it, new tickers simply wait for the nightly run instead.

10. If something breaks

Symptom	Likely cause and fix
Dashboard is blank	Opened index.html directly instead of via start_dashboard.bat — a browser cannot read neighbouring files from a bare file:// path.
Every row says NO FEED	signals.json has not been fetched yet. Check the Actions tab for a failed run, or trigger update-signals manually.
One stock says NO FEED, the rest are fine	Yahoo cannot resolve that ticker under either .NS or .BO. Add a YfSymbol override for it.
Nightly job fails to push	Settings → Actions → General → Workflow permissions must be Read and write.
Prices look stale on the hosted link	GitHub's CDN caches pages for a few minutes; force a fresh Pages build, or simply wait for the cache to expire on its own.
Weights look 100× wrong after import	A column mixed percent and fraction notation. The importer decides the convention once per column from its total — keep one convention per column.
A holdings upload is rejected outright	The Ticker column was not read correctly — the importer refuses to proceed if what it parsed as tickers contain spaces, which a real ticker never does.
GitHub save fails with a not-found error	Almost always the Branch field — it defaults to main, but this repository's branch is master.